DÉMO 004-E · CODE COMPLET

Error Boundary (version class)

Tous les fichiers utilisés, en entier, dans l'ordre de lecture.

Un Error Boundary en class component (getDerivedStateFromError, componentDidCatch) autour d'une page qui lance une erreur au clic sur « Throw error ».

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : react-error-boundary ^6.0.0

FICHIER **src/main.jsx** 9 lignes

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

FICHIER **src/App.jsx** 14 lignes

```jsx
import Home from "./components/Home";
import ErrorBoundary from "./components/ErrorBoundary";

import "./App.css";

function App() {
  return (
    <ErrorBoundary>
      <Home />
    </ErrorBoundary>
  );
}

export default App;
```

FICHIER **src/App.css** 39 lignes

```css
/* VERSION RESPONSIVE - S'adapte à tous les écrans (mobile, tablette, desktop) */

/* Importation de la police Google Fonts Inter avec plusieurs graisses */
@import url("https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap");

/* Sélecteur universel qui cible tous les éléments HTML */
* {
  /* Force tous les éléments à inclure padding et border dans leur largeur/hauteur totale */
  box-sizing: border-box;
  /* Supprime les marges par défaut du navigateur pour avoir un contrôle total */
  margin: 0;
  /* Supprime les espacements internes par défaut du navigateur */
  padding: 0;
}

/* Style global du corps de la page */
body {
  /* Applique la police Inter importée, avec sans-serif comme police de secours */
```

```
  font-family: "Inter", sans-serif;
  /* Améliore le rendu des polices sur les navigateurs WebKit (Safari, Chrome) */
  -webkit-font-smoothing: antialiased;
  /* Améliore le rendu des polices sur Firefox pour Mac */
  -moz-osx-font-smoothing: grayscale;

  text-align: center;

  padding: 20px;
}
.container {
  margin: 10px;
}

h1 {
  padding: 20px;
}

.page {
  margin: 20px;
}
```

FICHIER **src/components/Home.jsx** 18 lignes

```
import { useState } from "react";
function Home() {
  const [crash, setCrash] = useState(false);

  if (crash) {
    // Erreur lancée pendant le render : c'est ce que l'Error Boundary attrape (pas les erreurs d'un onClick)
    throw new Error("Test Error (render)");
  }

  return (
    <div>
      <h1>Welcome to the Login Page</h1>
      <button onClick={() => setCrash(true)}>Throw error</button>
    </div>
  );
}

export default Home;
```

FICHIER **src/components/ErrorBoundary.jsx** 38 lignes

```
import React from "react";

class ErrorBoundary extends React.Component {
  state = { hasError: false };

  // Appelée quand un enfant plante pendant le render : hasError passe à true, on affiche le fallback
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  componentDidCatch(error, info) {
    // Ici tu peux envoyer vers Sentry / LogRocket etc.
    console.error("ErrorBoundary caught:", error, info);
  }

  render() {
    if (this.state.hasError) {
      // fallback custom si tu veux
```

```
      if (this.props.fallback) return this.props.fallback;

      // fallback par défaut
      return (
        <div style={{ padding: 16 }}>
          <h2>Un problème est survenu</h2>
          <p>Merci de recharger la page ou de réessayer.</p>

          {/* Optionnel: afficher le message en dev */}
          {this.state.error && (
            <pre style={{ marginTop: 12 }}>{this.state.error.message}</pre>
          )}
        </div>
      );
    }

    return this.props.children;
  }
}
export default ErrorBoundary;
```